LabTrack - Service Layer Design

1. Overview

In LabTrack, user interface pages will never talk to the database directly. Every request goes through the service layer. The service layer is built with Spring Boot and Spring MVC and handles all business logic, including the status workflow enforcement and the auto-flagging of results. This document describes every endpoint the UI needs for the MVP.

All endpoints follow RESTful conventions. HTTP verbs are used consistently: GET retrieves data, POST creates new records, PUT updates existing ones. Responses are in JSON. Authentication is managed by Spring Security, and all endpoints except /login and /register require a valid session.

Method	Path	Purpose
POST	/login	Authenticate a user and create a session
POST	/register	Create a new user account
GET	/samples	Get all samples
GET	/test-types	Get the list of available test types
POST	/samples	Register a new patient sample
GET	/samples/{id}	Get a single sample and its result if available
PUT	/samples/{id}/status	Advance a sample's status one step forward
POST	/samples/{id}/result	Submit a result, flag auto-assigned by service layer

2. Endpoint Definitions

2.1 POST /login

Sam opens LabTrack and enters her username and password. The login form submits to this endpoint. Spring Security verifies the credentials against the users table. On success, a session is created and she is redirected to the dashboard. On failure, the login page reloads with an error message.

Request

```
POST /login
Content-Type: application/x-www-form-urlencoded
username=sam_j&password=mypassword
```

Success Response - 302 Redirect

```
HTTP/1.1 302 Found
Location: /dashboard
```

Error Response - 401 Unauthorized

```
HTTP/1.1 401 Unauthorized
{
  "error": "Invalid username or password."
}
```

2.2 POST /register

Sam or any new user creates an account. The registration form submits full_name, username, and password. The service layer checks whether the username already exists. If it does, it returns an error. If not, it hashes the password with BCrypt and inserts the new user into the users table.

Request

```
POST /register
{
  "fullName": "Sam J.",
  "username": "sam_j",
  "password": "mypassword"
}
```

Success Response - 201 Created

```
{
  "message": "Account created successfully. Please log in."
}
```

Error Response - 409 Conflict (duplicate username)

```
{
  "error": "Username 'sam_j' is already taken. Please choose another."
}
```

Error Response - 400 Bad Request (missing fields)

```
{
  "error": "All fields are required.",
  "fields": ["fullName", "password"]
}
```

2.3 GET /samples

When Sam or Dr. Peter lands on the dashboard, the page calls this endpoint to load the sample list. Dr. Peter uses the optional query parameters to filter the list. For example, he can filter to show only flagged results by passing flag=abnormal. The endpoint returns a summary object with stat card counts and the filtered sample list.

Request - default (no filter)

```
GET /samples
```

Request - with filters

```
GET /samples?status=COMPLETED&flag=abnormal
```

Success Response - 200 OK

```
{
  "stats": {
    "total": 42,
    "pending": 8,
    "completedToday": 34,
    "flagged": 3
  },
  "samples": [
```

```
    {
      "sampleId": 1,
      "patientName": "John Doe",
      "testType": "Glucose",
      "collectionDate": "2026-03-10",
      "status": "COMPLETED",
      "flag": "HIGH"
    },
    {
      "sampleId": 2,
      "patientName": "Jane Smith",
      "testType": "CBC",
      "collectionDate": "2026-03-10",
      "status": "IN_PROGRESS",
      "flag": null
    }
  ]
}
```


Error Response - 401 Unauthorized (not logged in)


```
{
  "error": "You must be logged in to access this resource."
}
```


2.4 GET /test-types

When Sam opens the New Sample form, the Test Type dropdown is populated by calling this endpoint. The service layer reads all rows from the test_types table and returns them as a list. This is needed so the form always reflects the current test catalog without hardcoding values in the UI.

Request

```
GET /test-types
```

Success Response - 200 OK


```
[
  { "testTypeId": 1, "name": "Glucose",     "unit": "mg/dL" },
  { "testTypeId": 2, "name": "CBC",         "unit": "g/dL"  },
  { "testTypeId": 3, "name": "Lipid Panel",  "unit": "mg/dL" },
  { "testTypeId": 4, "name": "Thyroid TSH",  "unit": "mIU/L" }
]
```


2.5 POST /samples

Sam fills in the New Sample form and clicks Register Sample. This endpoint creates a new row in the samples table. The service layer automatically sets the status to RECEIVED and records the logged-in user's ID as created_by. The patient name, test type, and collection date are required. Notes are optional. On success, Sam is redirected to the new sample's detail page.

Request

```
POST /samples
{
```

```
  "patientName": "John Doe",
  "testTypeId": 1,
  "collectionDate": "2026-03-10",
  "notes": "Fasting sample"
}
```

Success Response - 201 Created

```
{
  "sampleId": 5,
  "patientName": "John Doe",
  "testType": "Glucose",
  "collectionDate": "2026-03-10",
  "status": "RECEIVED",
  "createdBy": "sam_j",
  "createdAt": "2026-03-10T08:45:00"
}
```

Error Response - 400 Bad Request (missing required fields)

```
{
  "error": "Validation failed.",
  "fields": {
    "patientName": "Patient name is required.",
    "collectionDate": "Collection date is required."
  }
}
```

2.6 GET /samples/{id}

When Sam or Dr. Peter clicks on a row in the dashboard table, they are taken to the sample detail page. This endpoint fetches the full sample record plus the result if one has been entered. If no result exists yet, the result field is null. The page uses this data to decide what to show. For example, the result entry form is only enabled when status is COMPLETED.

Request

```
GET /samples/5
```

Success Response - 200 OK (before result)

```
{
  "sampleId": 5,
  "patientName": "John Doe",
  "testType": { "testTypeId": 1, "name": "Glucose", "unit": "mg/dL",
                "referenceMin": 70.0, "referenceMax": 140.0 },
  "collectionDate": "2026-03-10",
  "status": "RECEIVED",
  "notes": "Fasting sample",
  "createdBy": "sam_j",
  "result": null
}
```

Success Response - 200 OK (after result entered)

```
{
```

```
  "sampleId": 5,
  "patientName": "John Doe",
  "testType": { "testTypeId": 1, "name": "Glucose", "unit": "mg/dL",
                "referenceMin": 70.0, "referenceMax": 140.0 },
  "collectionDate": "2026-03-10",
  "status": "COMPLETED",
  "result": {
    "value": 185.0,
    "flag": "HIGH",
    "enteredAt": "2026-03-10T14:32:00"
  }
}
```


Error Response - 404 Not Found


```
{
  "error": "Sample with ID 99 was not found."
}
```


2.7 PUT /samples/{id}/status

Sam clicks the 'Mark as In Progress' or 'Mark as Completed' button on the sample detail page. This endpoint advances the sample's status by one step. The service layer enforces the one-directional workflow rule RECEIVED can only go to IN_PROGRESS, and IN_PROGRESS can only go to COMPLETED. Any other transition is rejected. The requested next status is sent to the request body.

Request


```
PUT /samples/5/status
{
  "status": "IN_PROGRESS"
}
```


Success Response - 200 OK


```
{
  "sampleId": 5,
  "previousStatus": "RECEIVED",
  "newStatus": "IN_PROGRESS",
  "updatedAt": "2026-03-10T09:15:00"
}
```


Error Response - 409 Conflict (invalid transition)


```
{
  "error": "Invalid status transition.",
  "detail": "Cannot change status from RECEIVED to COMPLETED. Next valid status is IN_PROGRESS."
}
```


Error Response - 400 Bad Request (already completed)


```
{
  "error": "This sample is already Completed. No further status changes are allowed."
}
```

2.8 POST /samples/{id}/result

Once a sample is Completed, Sam enters the numeric result value and clicks Submit. This endpoint receives the value, fetches the reference range for that test type from the test_types table, compares the value against reference_min and reference_max, assigns the appropriate flag (HIGH, LOW, or NORMAL), and writes both the value and the flag to the results table. Sam never manually selects the flag and the service layer decides it. Dr. Peter will see this flag on the dashboard immediately after it is written.

Request

```
POST /samples/5/result
{
  "value": 185.0
}
```

Success Response - 201 Created

```
{
  "resultId": 1,
  "sampleId": 5,
  "value": 185.0,
  "flag": "HIGH",
  "referenceRange": "70.0 - 140.0 mg/dL",
  "enteredAt": "2026-03-10T14:32:00"
}
```

Error Response - 409 Conflict (result already exists)

```
{
  "error": "A result has already been submitted for this sample. Results cannot be changed."
}
```

Error Response - 400 Bad Request (sample not completed)

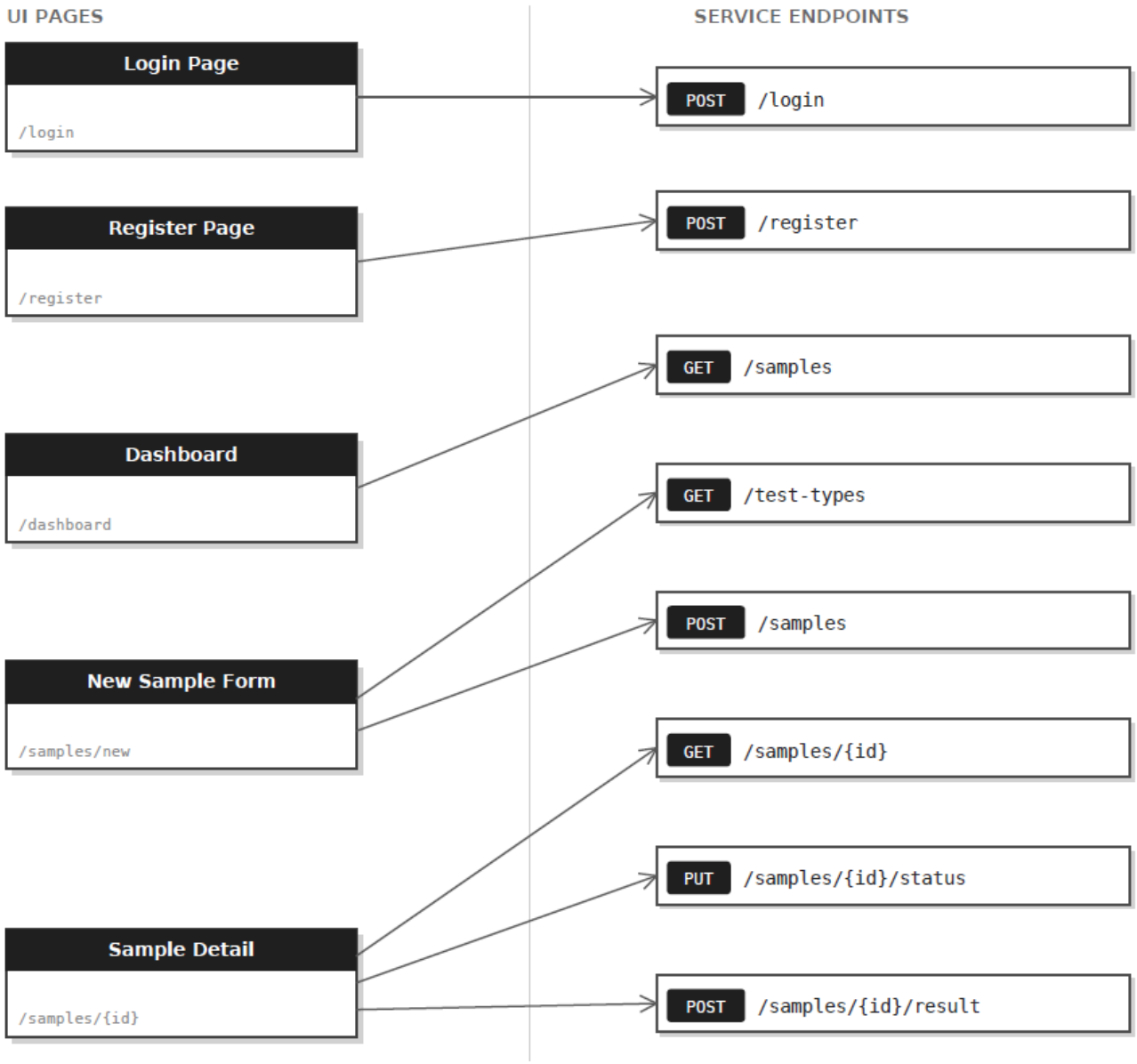
```
{
  "error": "Result cannot be submitted.",
  "detail": "Sample status must be COMPLETED before a result can be entered. Current status:
IN_PROGRESS."
}
```

Error Response - 400 Bad Request (invalid value)

```
{
  "error": "Result value must be a positive number."
}
```

3. UI to Endpoint Diagram

The diagram below shows which UI pages communicate with which service endpoints. Every arrow represents an HTTP request made from that page to that endpoint.



Notes on the diagram above:

The Dashboard page makes one call to GET /samples on load. If the user applies a filter, the same endpoint is called again with query parameters.

The New Sample Form loads the test type dropdown by calling GET /test-types on page load, then submits to POST /samples on form submission.

The Sample Detail page calls GET /samples/{id} on load. Depending on the current status, the user can then call PUT /samples/{id}/status or POST /samples/{id}/result.